Introduction to Software Engineering

Project Proposal & Plan



Software Engineering Department
Faculty of Information and Technology
University of Science

Table of Contents

1 Member Contribution Assessment..... 3
2 Vision..... 4
2.1 Preliminary Problem Statement..... 4
2.2 Product Position Statement..... 4
2.3 Stakeholders and Users..... 4
2.4 Scope and Key Features..... 5
3 Proposed Solution..... 6
3.1 Preliminary Architecture..... 6
3.2 Technology Stack..... 6
3.3 Hardware and Operating Environment..... 6
4 Project Plan..... 8
4.1 Process and Phase Gates..... 8
4.2 Work Breakdown Structure (WBS)..... 9
4.3 Estimation..... 10
4.4 Schedule..... 14
4.5 Roles and Responsibilities..... 14
4.6 Human Resources and Cost Estimate..... 14
5 Risk Management..... 15
6 Tools & Collaboration Setup..... 16
7 AI Usage Log..... 17

1 Member Contribution Assessment

[List every member. Contribution (%) must add up to 100%. Evidence: links to tasks on the board, commits or pull requests.]

ID	Name	Role(s)	Contribution (%)	Evidence
24127552	Nguyễn Võ Nhật Thuận	Team lead / Business analyst / Full-stack developer	20%	https://github.com/thuan65/introduction-to-software-engineer/issues/13 https://github.com/thuan65/introduction-to-software-engineer/issues/14 https://github.com/thuan65/introduction-to-software-engineer/issues/15
24127232	Lê Phụng Sơn	Business analyst / Team lead backup / Full-stack developer	20%	https://github.com/thuan65/introduction-to-software-engineer/issues/5 https://github.com/thuan65/introduction-to-software-engineer/issues/6 https://github.com/thuan65/introduction-to-software-engineer/issues/19 https://github.com/thuan65/introduction-to-software-engineer/issues/20
24127087	Dương Nguyên Nam	Designer / Full-stack developer / Business Analyst	20%	https://github.com/thuan65/introduction-to-software-engineer/issues/7 https://github.com/thuan65/introduction-to-software-engineer/issues/8 https://github.com/thuan65/introduction-to-software-engineer/issues/9
24127106	Lai Phạm Như Phúc	Full-stack developer / Tester	20%	https://github.com/thuan65/introduction-to-software-engineer/issues/16 https://github.com/thuan65/introduction-to-software-engineer/issues/17 https://github.com/thuan65/introduction-to-software-engineer/issues/18
24127086	Trần Hoàng Khánh My	Tester / Designer / Full-stack developer	20%	https://github.com/thuan65/introduction-to-software-engineer/issues/10 https://github.com/thuan65/introduction-to-software-engineer/issues/11

				https://github.com/thuan65/introduction-to-software-engineer/issues/12

2 Vision

2.1 Preliminary Problem Statement

SmartLib Booking

1. Background & Motivation

Managing study rooms, group discussion areas, and quiet spaces across university libraries is getting harder due to high student demand. Right now, our room booking process relies heavily on manual work. Students submit their booking requests using Google Docs, and librarians have to manually update and track schedules on a shared Google Sheet. This process is slow, prone to human error, and creates unnecessary delays for both students and library staff.

2. Core Problem & System Overview

To solve this, we are building **SmartLib Booking**, a web application designed to automate room reservations for a university library. The system replaces manual tracking with an automated scheduling workflow.

The software contains two main modules:

- **Student Module:** Students can search for available study rooms based on capacity, location, and equipment (like whiteboards or projectors). They can book rooms using their student ids, view the upcoming slots, and cancel or reschedule bookings.
- **Librarian & Staff Module:** Library staff can manage room inventories by adding, editing, or disabling room profiles. They can also set room status for maintenance.

The application supports two reservation workflows:

- **Manual Review Mode:** Student requests enter a queue for staff to review and approve manually.
- **Automated First-Come-First-Served Mode:** Librarians can toggle an auto-approval switch. During peak study times, the system automatically checks for schedule conflicts and approves valid requests instantly without staff intervention.
- **Notification System:** The system automatically sends email alerts whenever a booking is submitted, approved, or canceled.
- **Local AI Assistant (Optional):** Librarians can use an AI agent powered by a Vision-Language Model (VLM) running locally on their machine to search for rooms using natural language or answer questions about visual floor plans.

3. Operating Environment

- **Client Side:** Users access the web application through standard web browsers supporting HTML5 on both desktop and mobile devices.
- **Server & AI Runtime:** The backend runs on a Python web server host. The local VLM AI assistant runs via a lightweight API wrapper directly on the librarian's workstation hardware.

4. Design & Implementation Constraints

- **Tech Stack:** Built with Python and Django on the backend, using Django Templates and Tailwind CSS for the frontend UI.
- **Database:** Uses PostgreSQL for data storage, hosted on Supabase Cloud in production and running inside a Docker container during local development.
- **Project Scope:** The project is strictly single-tenant, designed for one library system (such as VNU Central Library). It is restricted to a web application without native mobile apps. Online payment processing or fee settlements are out of scope.
- **Non-Functional Constraints:** The local VLM must run within reasonable GPU/RAM limits on standard staff workstations to prevent system crashes. All user access requires valid student or staff account authentication.

2.2 Product Position Statement

Element	Content
For	university students and library staff at a single university library (e.g., VNU Central Library)
Who	need a fast, reliable way to find and reserve study rooms, and currently depend on Google Docs forms and a shared Google Sheet that librarians must update and check by hand
The SmartLib Booking	web-based study-room reservation system
That	lets students browse rooms, book online and manage their reservations, while librarians manage room inventory and approvals with conflict detection, an optional automatic first-come-first-served mode, email notifications, and an AI assistant for natural-language room search
Unlike	LibCal (feature-heavy and still reliant on staff reviewing requests), Skedda (built for commercial and corporate workspaces), and the current manual Google Docs/Sheets process
Our product	is lightweight and designed for non-commercial academic libraries. It removes the manual approval queue at peak times and offers AI-assisted search through a locally hosted Vision-Language Model, so there are no cloud AI costs.

2.3 Stakeholders and Users

#	Stakeholder / user	Description	Main interests
1	Project sponsor (course instructor & TA)	Evaluate the project and the team's teamwork and process	A complete, well-documented product that meets the PA requirements; correct use of tools and the AI policy

2	Student (end user)	University students who book rooms for individual or group study	Seeing room availability and equipment quickly, booking with instant confirmation, managing bookings (view, cancel, reschedule), receiving reminders and status emails
3	Librarian / library staff (end user)	Staff who manage rooms and process booking requests	Managing room profiles (add, edit, disable, maintenance status), no double bookings, less manual tracking, a toggle for auto-approval
4	Library management	Decision makers responsible for library services and policies	Better room utilization, fewer scheduling conflicts, lower staff workload, low operating cost
5	Development team (TPNSM)	Five students who design, build, test and deploy the system	A feasible scope, a manageable workload, a working demo, and a target grade of 9.0 or higher

2.4 Scope and Key Features

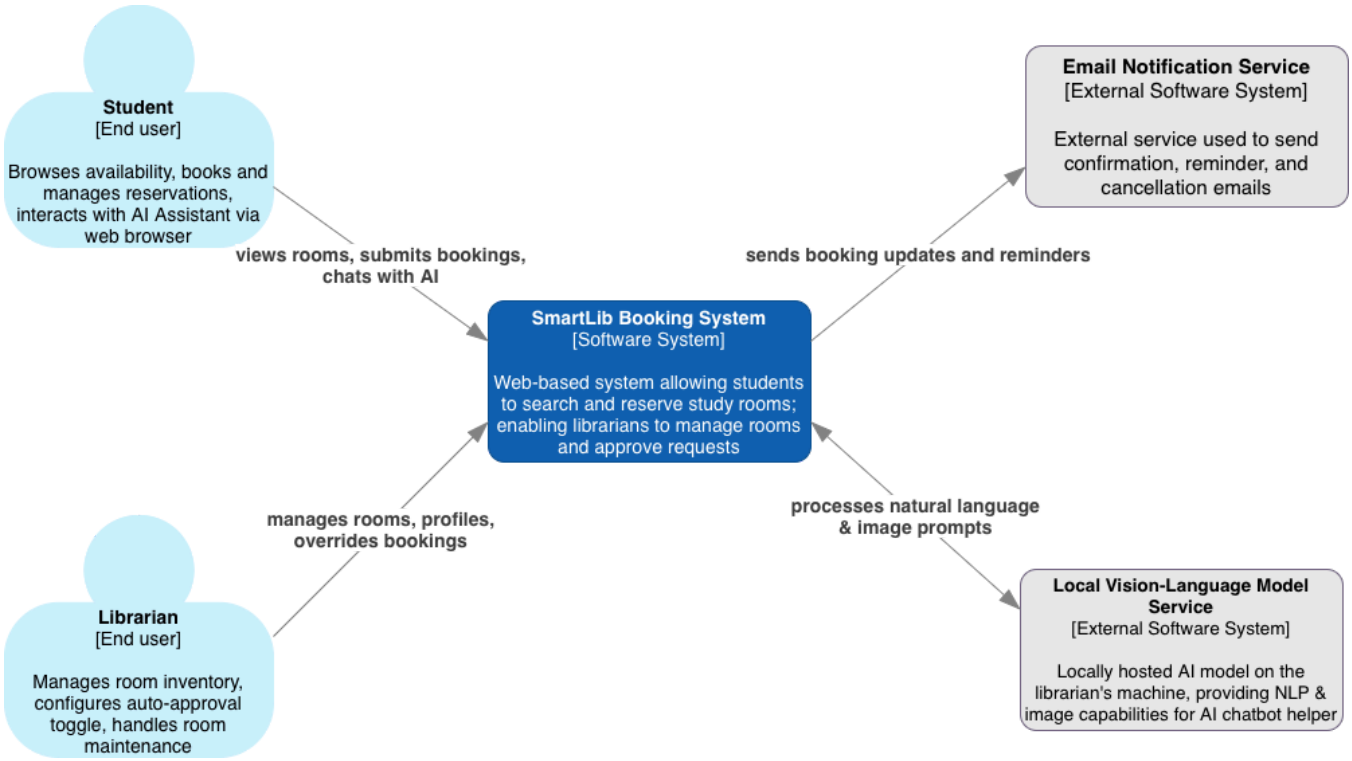
ID	Feature	Description	Priority (M/S/C/W)
F01	Basic Room Booking	Students view available rooms by date, time and attributes, then submit a reservation request to the librarian for approval. The system prevents conflicting bookings.	Must
F02	Room Inventory Management	Librarians add, edit and disable rooms, and set attributes such as location, capacity, amenities and maintenance status.	Must
FO3	Automated First-Come-First-Served Mode	A librarian toggle that approves incoming requests automatically, in order of arrival, when no conflict exists.	Should
FO4	AI Assistant	A chat assistant powered by a locally hosted VLM for natural-language room search, floor plan questions and booking help.	Should
FO5	Automated Status Notifications	Email notifications when a booking is requested, approved or canceled.	Should

Out of scope:

- Online payment or fine settlement (no payment gateway for booking fees or late penalties)
- Native mobile apps (web application only, no App Store or Play Store builds)
- Multi-library or multi-campus support (a single university library only)
- Cloud-hosted AI services (the AI assistant runs locally only)

3 Proposed Solution

3.1 Preliminary Architecture



3.2 Technology Stack

Layer	Technology	Reason for choice
Front end	Django Templates, Tailwind CSS	Using Django Templates enables Server-Side Rendering (SSR), while Tailwind CSS allows for rapid UI development without writing complex raw CSS files.
Back end	Python	Python provides a robust ecosystem and rich libraries for logic processing. The framework includes a built-in ORM and a static Admin panel, accelerating the development of room management features and booking workflows.
Database	PostgreSQL	PostgreSQL is a powerful relational database management system that ensures data integrity (ACID) and efficiently handles complex queries to prevent booking scheduling conflicts.
Hosting / deployment	Supabase Cloud (Database), Web Hosting Platform (Render/Railway)	Supabase Cloud provides a stable and easy-to-configure PostgreSQL hosting solution for the production environment.
Testing & CI	Docker, GitHub Actions, PyTest	The local testing environment is containerized using Docker to ensure consistency across machines. GitHub Actions automates the source code testing workflow.

3.3 Hardware and Operating Environment

- Development Environment:
    + Hardware: Personal computers (PC/Laptop) of team members running Windows, macOS or Linux.
    + Software & Tools: Requires Python and Docker Desktop. Each member will set up their Local Development Environment by spinning up a Docker Container running an independent PostgreSQL instance. This ensures database synchronization across personal machines without needing to connect to the cloud database during development.
- Operating Environment (Production):
    + The system is deployed as a Web Application.
    + The production database is hosted and operated on Supabase Cloud.
    + The backend server requires an environment that supports Python code execution.
- Client Environment:
    + End users (Students and Librarians) only need a modern web browser (Google Chrome, Safari, Edge, Firefox) on a computer or mobile device with an internet connection to access the web application. No additional software installation is required.

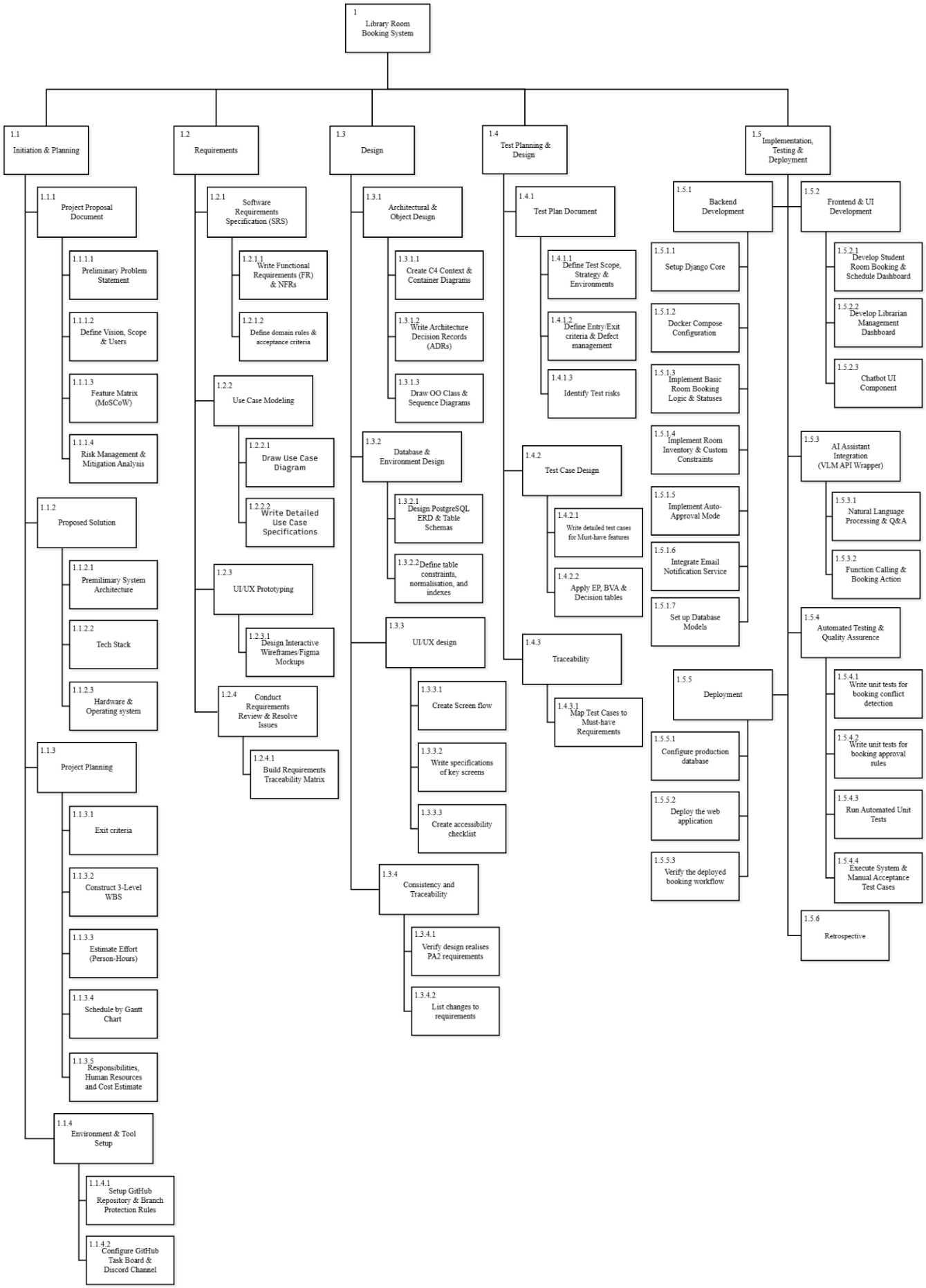
4 Project Plan

4.1 Process and Phase Gates

The project follows the waterfall model. Each phase ends with a phase gate (a PA submission). After a PA is submitted it becomes a baseline; later changes must be updated in the document and listed in the next PA.

PA	Phase	Due	Exit criteria
PA1	Initiation & planning	End of week 3	The project proposal and plan are completed (vision and scope) with MoSCoW MoSCoW, a WBS with at least three levels, estimation, a Gantt chart with dependencies and milestones and owners; and the required repository, task board and communication channel setup.
PA2	Requirements	End of week 5	The Software Requirements Specification (SRS) document is completed, including FRs/NFRs, use case model and specifications, UI prototype, and requirements traceability matrix.
PA3	Design	End of week 7	The software design completed, including architecture diagrams, at least three Architecture Decision Records, object-oriented design diagrams and specifications, data design, UI/UX design, and traceability to PA2 requirements.
PA4	Test planning	End of week 8	The test plan and test cases are completed, including test scope, techniques, environment, entry and exit criteria, and traceability of all Must-have requirements to test cases.
PA5	Implementation, testing & deployment	End of week 10	The Must-have features are implemented and accessible. Test , execution results, code reviewed.
Final	Presentation & demo	Week 11	The presentation and live demonstration are prepared. The team presents the required project deliverables, demonstrates working scenarios.

4.2 Work Breakdown Structure (WBS)



4.3 Estimation

[Estimate the effort of each work package (person-hours) and explain the method used (expert judgement, three-point estimate, analogy).]

WBS ID	Work package	Owner	Effort (person-hours)	Method / assumptions
1.1	**PA01 – Initiation & planning**			
1.1.1.1	Preliminary Problem Statement	Lê Phụng Sơn	2	Expert Judgement
1.1.1.2	Define Vision, Scope & Target Users	Dương Nguyên Nam	2.16	Three-Point (O=1, M=2, P=4 → (1+8+4)/6 = 2.16h)
1.1.1.3	Draft Feature Matrix (MoSCoW)	Dương Nguyên Nam	2	Analogy
1.1.1.4	Conduct Risk Management & Mitigation Analysis	Lê Phụng Sơn	8	Analogy
1.1.2.1	Preliminary System Architecture	Khánh My	4.16	Three-Point (O=3, M=4, P=6→ (3+16+6)/6 = 4.16h)
1.1.2.2	Tech Stack	Khánh My	3	Analogy
1.1.2.3	Hardware & Operating system	Khánh My	3	Analogy
1.1.3.1	Exit criteria	Nguyễn Võ Nhật Thuận	2	Analogy
1.1.3.2	Construct 3-Level WBS	Nguyễn Võ Nhật Thuận	4.8	Three-Point (O=3, M=5, P=6→ (3+20+6)/6 = 4.8h)
1.1.3.3	Estimate Effort (Person-Hours)	Nguyễn Võ Nhật Thuận	8	Analogy
1.1.3.4	Schedule by Gantt Chart	Lai Phạm Như Phúc	4	Analogy
1.1.3.5	Responsibilities, Human Resources and Cost Estimate	Lai Phạm Như Phúc	3	Analogy
1.1.4.1	Setup GitHub Repository & Branch Protection Rules	Nguyễn Võ Nhật Thuận	2	Analogy

1.1.4.2	Configure GitHub Task Board & Discord Channel	Nguyễn Võ Nhật Thuận	2	Analogy
1.2	**PA2 – Requirements**			
1.2.1.1	Write Functional Requirements (FR) & NFRs	Lê Phụng Sơn	6.8	Three-Point (O=5, M=7, P=8 → (5+28+8)/6 = 6.8h)
1.2.1.2	Define domain rules & acceptance criteria	Nguyễn Võ Nhật Thuận	3.1	Three-Point (O=2, M=3, P=5 → (2+12+5)/6 = 3.1h)
1.2.2.1	Draw Use Case Diagram	Dương Nguyên Nam	3	Expert Judgement
1.2.2.2	Write Detailed Use Case Specifications	Nguyễn Võ Nhật Thuận	3.1	Three-Point (O=2, M=3, P=5 → (2+12+5)/6 = 3.1h)
1.2.3.1	Design Interactive Wireframes/Figma Mockups	Lai Phạm Như Phúc	5.1	Three-Point (O=3, M=5, P=8 → (3+20+8)/6 = 5.1h)
1.2.4.1	Build Requirements Traceability Matrix	Trần Hoàng Khánh My	3	Analogy (Based on a standard RTM template)
1.3	**PA3 – Design**			
1.3.1.1	Create C4 Context & Container Diagrams	Lê Phụng Sơn	4.3	Three-Point (O=3, M=4, P=6 → (3+4+6)/3 = 4.3h)
1.3.1.2	Write Architecture Decision Records (ADRs)	Nguyễn Võ Nhật Thuận	3	Expert Judgement
1.3.1.3	Draw OO Class & Sequence Diagrams	Lai Phạm Như Phúc	3.1	Three-Point (O=2, M=3, P=5 → (2+12+5)/6 = 3.1h)
1.3.2.1	Design PostgreSQL ERD & Table Schemas	Dương Nguyên Nam	3.8	Three-Point (O=2, M=4, P=5 → (2+12+5)/6 = 3.8h)
1.3.2.2	Define table constraints, normalisation, and indexes	Dương Nguyên Nam	2	Expert Judgement
1.3.3.1	Create Screen flow	Lai Phạm Như Phúc	2	Analogy

1.3.3.2	Write specifications of key screens	Lê Phụng Sơn	1.5	Expert Judgement
1.3.3.3	Create accessibility checklist	Lê Phụng Sơn	2	Analogy
1.3.4.1	Verify design realises PA2 requirements	Dương Nguyên Nam	3	Expert Judgement
1.3.4.2	List changes to requirements	Dương Nguyên Nam	2	Expert Judgement
1.4	**PA4 - Test planning**			
1.4.1.1	Define Test Scope, Strategy & Environments	Trần Hoàng Khánh My	3.1	Three-Point (O=2, M=3, P=5 → (2+12+5)/6 = 3.1h)
1.4.1.2	Define Entry/Exit criteria & Defect management	Trần Hoàng Khánh My	2	Analogy
1.4.1.3	Identify Test risks	Nguyễn Võ Nhật Thuận	2	Expert Judgement
1.4.2.1	Write detailed test cases for Must-have features	Lai Phạm Như Phúc	3.8	Three-Point (O=2, M=4, P=5 → (2+12+5)/6 = 3.8h)
1.4.2.2	Apply EP, BVA & Decision tables	Trần Hoàng Khánh My	3	Expert Judgement
1.4.3.1	Map Test Cases to Must-have Requirements	Nguyễn Võ Nhật Thuận	3	Analogy
1.5	**PA5 - Implementation, testing & deployment**			
1.5.1.1	Set up Django Core	Lê Phụng Sơn	2	Analogy
1.5.1.2	Docker Compose Configuration	Lê Phụng Sơn	2	Expert Judgement
1.5.1.3	Implement Basic Room Booking Logic & Statuses	Lai Phạm Như Phúc	3.8	Three-Point (O=2, M=4, P=5 → (2+12+5)/6 = 3.8h)

1.5.1.4	Implement Room Inventory & Custom Constraints	Lê Phụng Sơn	3	Three-Point (O=2, M=3, P=4 → (2+12+4)/6 = 3h)
1.5.1.5	Implement Auto-Approval Mode	Lê Phụng Sơn	3	Expert Judgement
1.5.1.6	Integrate Email Notification Service	Nguyễn Võ Nhật Thuận	2	Analogy
1.5.1.7	Set up Database Models	Dương Nguyên Nam	2	Analogy
1.5.2.1	Develop Student Room Booking & Schedule Dashboard	Lai Phạm Như Phúc	3.8	Three-Point (O=2, M=4, P=5 → (2+12+5)/6 = 3.8h)
1.5.2.2	Develop Librarian Management Dashboard	Dương Nguyên Nam	3.1	Three-Point (O=2, M=3, P=5 → (2+12+5)/6 = 3.1h)
1.5.2.3	Chatbot UI Component	Dương Nguyên Nam	3	Analogy
1.5.3.1	Natural Language Processing & Q&A	Nguyễn Võ Nhật Thuận	3	Analogy
1.5.3.2	Function Calling & Booking Action	Nguyễn Võ Nhật Thuận	4.6	Three-Point (O=3, M=4, P=7 → (3+4+7)/3 = 4.6h)
1.5.4.1	Write unit tests for booking conflict detection	Lai Phạm Như Phúc	3	Expert Judgement
1.5.4.2	Write unit tests for booking approval rules	Lai Phạm Như Phúc	3	Expert Judgement
1.5.4.3	Run Automated Unit Tests	Trần Hoàng Khánh My	2	Analogy
1.5.4.4	Execute System & Manual Acceptance Test Cases	Trần Hoàng Khánh My	4	Three-Point (O=3, M=4, P=5 → (3+16+5)/6 = 4h)
1.5.5.1	Configure production database	Dương Nguyên Nam	2	Expert Judgement

1.5.5.2	Deploy the web application	Nguyễn Võ Nhật Thuận	2	Three-Point (O=1, M=2, P=4⟶ (1+8+4)/6 = 2.1h)
1.5.5.3	Verify the deployed booking workflow	Dương Nguyên Nam	2	Expert Judgement
1.5.6.1	Project Retrospective & Final Handover	Nguyễn Võ Nhật Thuận	2	Expert Judgement (The entire team will attend one meeting session)

4.4 Schedule

Gantt Chart: https://docs.google.com/spreadsheets/d/1zCB93u_6uJuVgFYFEFaGw8xFlX4H0bV6Ntd6rZJ0t8c/edit?usp=sharing

Project Title	SmartLib Booking
Team Leader	Nguyễn Võ Nhật Thuận
Team ID - Name	04 - TPNSM



WBS Number	Task Title	Task Owner	Start Date	Due Date	Duration (Days)	Predecessors (Dependencies)	% of Task Complete	Critical Path?
1.1	**PA01 - Initiation & planning (Weeks 1–3)**		**2026-10-01**	**2026-10-21**	**20**		**100%**	
1.1.1.1	Preliminary Problem Statement	Lê Phụng Sơn	2026-10-01	2026-10-04	3	-	100%	**Yes (CP)**
1.1.1.2	Define Vision, Scope & Target Users	Dương Nguyễn Nam	2026-10-04	2026-10-09	5	1.1.1.1	100%	**Yes (CP)**
1.1.4.1	Setup Git-hub Repository & Branch Protection Rules	Nguyễn Võ Nhật Thuận	2026-10-08	2026-10-11	3	-	100%	No
1.1.1.3	Draft Feature Matrix (MoSCoW)	Dương Nguyễn Nam	2026-10-09	2026-10-13	4	1.1.1.2	100%	**Yes (CP)**
1.1.2.1	Preliminary System Architecture	Trần Hoàng Khánh My	2026-10-09	2026-10-14	5	1.1.1.2	100%	No
1.1.4.2	Configure Git-hub Task Board & Discord Channel	Nguyễn Võ Nhật Thuận	2026-10-11	2026-10-13	2	1.1.4.1	100%	No
1.1.3.1	Exit criteria	Nguyễn Võ Nhật Thuận	2026-10-12	2026-10-14	2	1.1.1.2	100%	No
M0	**Milestone: PA0 Submission (Idea & Contract)**	**All Members**	**2026-10-14**	**2026-10-14**	**0**	**1.1.1.3, 1.1.4.2**	**100%**	**Milestone**
1.1.1.4	Conduct Risk Management & Mitigation Analysis	Lê Phụng Sơn	2026-10-13	2026-10-16	3	1.1.1.3	100%	No
1.1.3.2	Construct 3-Level WBS	Nguyễn Võ Nhật Thuận	2026-10-13	2026-10-16	3	1.1.1.3	100%	**Yes (CP)**
1.1.2.2	Tech Stack	Trần Hoàng Khánh My	2026-10-14	2026-10-17	3	1.1.2.1	100%	No
1.1.3.3	Estimate Effort (Person-Hours)	Nguyễn Võ Nhật Thuận	2026-10-16	2026-10-18	2	1.1.3.2	100%	**Yes (CP)**
1.1.2.3	Hardware & Operating system	Trần Hoàng Khánh My	2026-10-17	2026-10-19	2	1.1.2.2	100%	No
1.1.3.4	Schedule by Gantt Chart	Lại Phạm Như Phúc	2026-10-18	2026-10-21	3	1.1.3.3	100%	**Yes (CP)**
1.1.3.5	Responsibilities, Human Resources and Cost Estimate	Lại Phạm Như Phúc	2026-10-18	2026-10-20	2	1.1.3.3	100%	No
M1	**Milestone: PA1 Submission (Proposal & Plan)**	**All Members**	**2026-10-21**	**2026-10-21**	**0**	**1.1.3.4, 1.1.3.5, 1.1.1.4, 1.1.2.3**	**100%**	**Milestone (CP)**
1.2	**PA2 - Requirements (Weeks 4–5)**		**2026-10-21**	**2026-11-04**	**14**		**0%**	
1.2.1.1	Write Functional Requirements (FR) & NFRs	Lê Phụng Sơn	2026-10-21	2026-10-25	4	M1	0%	**Yes (CP)**
1.2.1.2	Define domain rules & acceptance criteria	Nguyễn Võ Nhật Thuận	2026-10-22	2026-10-25	3	M1	0%	No
1.2.2.1	Draw Use Case Diagram	Dương Nguyễn Nam	2026-10-25	2026-10-27	2	1.2.1.1	0%	**Yes (CP)**
1.2.2.2	Write Detailed Use Case Specifications	Nguyễn Võ Nhật Thuận	2026-10-27	2026-11-01	5	1.2.2.1, 1.2.1.2	0%	**Yes (CP)**
1.2.3.1	Design Interactive Wireframes/Figma Mockups	Lại Phạm Như Phúc	2026-10-27	2026-11-01	5	1.2.2.1	0%	No
1.2.4.1	Build Requirements Traceability Matrix	Trần Hoàng Khánh My	2026-11-01	2026-11-04	3	1.2.2.2, 1.2.3.1	0%	**Yes (CP)**
M2	**Milestone: PA2 Submission (SRS)**	**All Members**	**2026-11-04**	**2026-11-04**	**0**	**1.2.4.1**	**0%**	**Milestone (CP)**
1.3	**PA3 - Design (Weeks 6–7)**		**2026-11-04**	**2026-11-18**	**14**		**0%**	
1.3.1.1	Create C4 Context & Container Diagrams	Lê Phụng Sơn	2026-11-04	2026-11-08	4	M2	0%	**Yes (CP)**
1.3.1.2	Write Architecture Decision Records (ADRs)	Nguyễn Võ Nhật Thuận	2026-11-05	2026-11-08	3	M2	0%	No
1.3.2.1	Design PostgreSQL ERD & Table Schemas	Dương Nguyễn Nam	2026-11-04	2026-11-08	4	M2	0%	No
1.3.3.1	Create Screen flow	Lại Phạm Như Phúc	2026-11-04	2026-11-07	3	M2	0%	No
1.3.3.2	Write specifications of key screens	Lê Phụng Sơn	2026-11-07	2026-11-10	3	1.3.3.1	0%	No
1.3.1.3	Draw OO Class & Sequence Diagrams	Lại Phạm Như Phúc	2026-11-08	2026-11-13	5	1.3.1.1, 1.3.2.1	0%	**Yes (CP)**
1.3.2.2	Define table constraints, normalisation, and indexes	Dương Nguyễn Nam	2026-11-08	2026-11-12	4	1.3.2.1	0%	No
1.3.3.3	Create accessibility checklist	Lê Phụng Sơn	2026-11-10	2026-11-12	2	1.3.3.2	0%	No
1.3.4.1	Verify design realises PA2 requirements	Dương Nguyễn Nam	2026-11-13	2026-11-16	3	1.3.1.3, 1.3.2.2, 1.3.3.3	0%	**Yes (CP)**
1.3.4.2	List changes to requirements	Dương Nguyễn Nam	2026-11-16	2026-11-18	2	1.3.4.1	0%	**Yes (CP)**
M3	**Milestone: PA3 Submission (Software Design)**	**All Members**	**2026-11-18**	**2026-11-18**	**0**	**1.3.4.2**	**0%**	**Milestone (CP)**
1.4	**PA4 - Test planning (Week 8)**		**2026-11-18**	**2026-11-25**	**7**		**0%**	
1.4.1.1	Define Test Scope, Strategy & Environments	Trần Hoàng Khánh My	2026-11-18	2026-11-20	2	M3	0%	**Yes (CP)**
1.4.1.2	Define Entry/Exit criteria & Defect management	Trần Hoàng Khánh My	2026-11-18	2026-11-20	2	M3	0%	No
1.4.1.3	Identify Test risks	Nguyễn Võ Nhật Thuận	2026-11-19	2026-11-20	1	M3	0%	No
1.4.2.1	Write detailed test cases for Must-have features	Lại Phạm Như Phúc	2026-11-20	2026-11-24	4	1.4.1.1, 1.4.1.2	0%	**Yes (CP)**
1.4.2.2	Apply EP, BVA & Decision tables	Trần Hoàng Khánh My	2026-11-20	2026-11-23	3	1.4.1.1	0%	No
1.4.3.1	Map Test Cases to Must-have Requirements	Nguyễn Võ Nhật Thuận	2026-11-24	2026-11-25	1	1.4.2.1, 1.4.2.2	0%	**Yes (CP)**
M4	**Milestone: PA4 Submission (Test Plan & Cases)**	**All Members**	**2026-11-25**	**2026-11-25**	**0**	**1.4.3.1**	**0%**	**Milestone (CP)**
1.5	**PA5 - Implementation, testing & deployment (Weeks 9–11)**		**2026-11-25**	**2026-12-16**	**21**		**0%**	
1.5.1.1	Setup Django Core	Lê Phụng Sơn	2026-11-25	2026-11-26	1	M4	0%	**Yes (CP)**
1.5.1.2	Docker Compose Configuration	Lê Phụng Sơn	2026-11-25	2026-11-27	2	M4	0%	No
1.5.1.7	Set up Database Models	Dương Nguyễn Nam	2026-11-26	2026-11-27	1	1.5.1.1	0%	**Yes (CP)**
1.5.1.3	Implement Basic Room Booking Logic & Statuses	Lại Phạm Như Phúc	2026-11-27	2026-12-01	4	1.5.1.7	0%	**Yes (CP)**
1.5.1.4	Implement Room Inventory & Custom Constraints	Lê Phụng Sơn	2026-11-27	2026-11-30	3	1.5.1.7	0%	No
1.5.2.1	Develop Student Room Booking & Schedule Dashboard	Lại Phạm Như Phúc	2026-11-27	2026-12-01	4	1.5.1.7	0%	No
1.5.2.2	Develop Librarian Management Dashboard	Dương Nguyễn Nam	2026-11-27	2026-12-01	4	1.5.1.7	0%	No
1.5.3.1	Natural Language Processing & Q&A	Nguyễn Võ Nhật Thuận	2026-11-27	2026-11-30	3	1.5.1.2	0%	No
1.5.3.2	Function Calling & Booking Action	Nguyễn Võ Nhật Thuận	2026-11-30	2026-12-03	3	1.5.3.1, 1.5.1.3	0%	No
1.5.1.5	Implement Auto-Approval Mode	Lê Phụng Sơn	2026-12-01	2026-12-03	2	1.5.1.3	0%	No
1.5.1.6	Integrate Email Notification Service	Nguyễn Võ Nhật Thuận	2026-12-01	2026-12-03	2	1.5.1.3	0%	No
1.5.2.3	Chatbot UI Component	Dương Nguyễn Nam	2026-12-01	2026-12-03	2	1.5.2.1	0%	No
1.5.4.1	Write unit tests for booking conflict detection	Lại Phạm Như Phúc	2026-12-01	2026-12-03	2	1.5.1.3	0%	**Yes (CP)**
1.5.4.2	Write unit tests for booking approval rules	Lại Phạm Như Phúc	2026-12-03	2026-12-04	1	1.5.4.1, 1.5.1.5	0%	**Yes (CP)**
1.5.4.3	Run Automated Unit Tests	Trần Hoàng Khánh My	2026-12-04	2026-12-05	1	1.5.4.2	0%	**Yes (CP)**
1.5.4.4	Execute System & Manual Acceptance Test Cases	Trần Hoàng Khánh My	2026-12-05	2026-12-07	2	1.5.4.3, 1.5.2.1, 1.5.2.2	0%	**Yes (CP)**
1.5.5.1	Configure production database	Dương Nguyễn Nam	2026-12-05	2026-12-07	2	1.5.1.7	0%	No
1.5.5.2	Deploy the web application	Nguyễn Võ Nhật Thuận	2026-12-07	2026-12-08	1	1.5.4.4, 1.5.5.1	0%	**Yes (CP)**
1.5.5.3	Verify the deployed booking workflow	Dương Nguyễn Nam	2026-12-08	2026-12-09	1	1.5.5.2	0%	**Yes (CP)**
M5	**Milestone: PA5 Submission (Impl, Test & Deploy)**	**All Members**	**2026-12-09**	**2026-12-09**	**0**	**1.5.5.3**	**0%**	**Milestone (CP)**
1.5.6.1	Project Retrospective & Final Handover	Nguyễn Võ Nhật Thuận	2026-12-09	2026-12-16	7	M5	0%	**Yes (CP)**
M6	**Milestone: Final Presentation & Demo (Week 11)**	**All Members**	**2026-12-16**	**2026-12-16**	**0**	**1.5.6.1**	**0%**	**Milestone (CP)**

4.5 Roles and Responsibilities

Role	Main	Backup	Responsibilities
Team lead / Project manager	Nguyễn Võ Nhật Thuận	Lê Phụng Sơn	Plan, assign tasks, weekly status report, contact point
Business analyst	Lê Phụng Sơn	Dương Nguyên Nam	Elicit and document requirements, organise reviews
Designer / Architect	Dương Nguyên Nam	Trần Hoàng Khánh My	Architecture, class, data and UI design
Developers	Lai Phạm Như Phúc	Nguyễn Võ Nhật Thuận Lê Phụng Sơn Dương Nguyên Nam Trần Hoàng Khánh My	Implement, unit test, review pull requests
Tester	Trần Hoàng Khánh My	Lai Phạm Như Phúc	Test plan, test cases, system testing, test report

4.6 Human Resources and Cost Estimate

Project Duration: 11 weeks (from October 1, 2026 to December 16, 2026).

Total Estimated Effort: 173.12 person-hours across **56 work packages** (as detailed in Section 4.3 Estimation). This aligns closely with our Team Contract commitment of approximately 3.5 to 4.5 hours per member per week.

Hourly Labor Rate: We assume a realistic market rate for a **Junior Software Developer / Intern in Vietnam** at **$5.00 USD / person-hour** (equivalent to **125,000 VND / hour**, at an exchange rate of $1USD = 25,000VND$). This rate is applied uniformly across all five team members.

Infrastructure & Tooling Strategy: The team utilizes open-source frameworks, student developer packs, free-tier database hosting, and a **locally hosted Vision-Language Model (VLM)** to eliminate commercial cloud AI API fees.

Human Resources Cost Breakdown (By Team Member)

Student ID	Member Name	Main Role	Backup Role(s)	Assigned Tasks	Estimated Effort (Person-Hours)	Workload Share (%)	Hourly Rate (USD)	Labor Cost (USD)	Labor Cost (VND)
24127552	Nguyễn Võ Nhật Thuận	Team Lead / PM	BA, Full-stack Dev	15	46.6	26.92%	$5.00	$233.00	5,825,000
24127232	Lê Phụng Sơn	Business Analyst	Team Lead, Full-stack Dev	10	34.6	19.99%	$5.00	$173.00	4,325,000
24127087	Dương Nguyên Nam	Designer / Architect	BA, Full-stack Dev	12	30.06	17.36%	$5.00	$150.30	3,757,500
24127106	Lai Phạm Như Phúc	Full-stack Developer	Tester	10	34.6	19.99%	$5.00	$173.00	4,325,000
24127086	Trần Hoàng Khánh My	Tester / QA	Designer, Full-stack Dev	9	27.26	15.75%	$5.00	$136.30	3,407,500
Total	**5 Members**			**56**	**173.12 hrs**	**100.00%**	**$5.00**	**$865.60**	**21,640,000**

Tools, Hosting, Infrastructure and Other Costs (Non-Labor)

Category	Item / Service	Pricing Tier / Model	Duration	Monthly Cost	Total Cost (USD)	Total Cost (VND)	Notes / Justification
Development & Collaboration	GitHub (Repo, Projects, Actions CI), VS Code, Figma, Discord	Free / GitHub Student Developer Pack	3 months	$0.00	$0.00	0	Free academic/education licenses for university students.

Local Environment & AI	Docker Desktop, Python/Django, Local VLM (API Wrapper)	Open-Source / Local Execution	3 months	$0.00	$0.00	0	VLM runs on local workstation hardware, incurring zero cloud AI costs.
Database Hosting	Supabase Cloud (PostgreSQL Production DB)	Free Tier (up to 500 MB DB)	2 months	$0.00	$0.00	0	Sufficient for single-library booking records and PA5 evaluation.
Web Application Hosting	Render / Railway (Web Service Starter Plan)	$7.00 / month (Starter instance)	2 months (W9–W11)	$7.00	$14.00	350,000	Prevents cold-start latency during PA5 testing and live demo grading.
Domain & Email Service	Custom Domain + Standard SMTP Service	GitHub Student Pack / Free SMTP	3 months	$0.00	$0.00	0	Free domain via student pack; standard SMTP for email alerts.
Other Operational Costs	Local GPU/Electricity & Printing PA Deliverables	Estimated flat allowance	3 months	-	$16.00	400,000	Covers local VLM runtime power consumption and document printing.
Subtotal					**$30.00**	**750,000**	

Total Project Cost Summary

Cost Component	Calculation / Basis	Total (USD)	Total (VND)	Percentage (%)
1. Direct Labor Cost	173.12 person-hours x $5.00/hour	$865.60	21,640,000	87.86%
2. Tools, Hosting & Other Costs	Cloud web hosting ($14.00) + Operational/Electricity ($16.00)	$30.00	750,000	3.05%
3. Base Project Cost (1 + 2)	Direct Labor + Non-Labor Costs	**$895.60**	**22,390,000**	**90.91%**
4. Contingency Reserve (10%)	10% of Base Cost (Risk buffer for PA5 bug fixing & rework)	$89.56	2,239,000	9.09%
Total Estimated Project Budget	**Base Project Cost + Contingency Reserve**	**$985.16**	**24,629,000**	**100.00%**

5 Risk Management

Loss scale reference:

- 1 — Negligible (minimal effect on outcome)
- 2 — Marginal (minor impact / easily workaroundable)
- 3 — Critical (major feature broken / severe delay)
- 4 — Catastrophic (project failure / core goals unfulfilled)

ID	Risk	Type	Probability	Loss	RE	Mitigation / contingency	Owner
R1	High Resource Consumption and Response Latency from Local VLM Execution	Technology	40%	3	1.2	Pre-test model memory consumption on baseline hardware; implement a fallback text-only keyword search if the vision-language model fails or lags	Full-stack developer
R2	Simultaneous Booking Requests Causing Concurrency and Double-Booking Conflicts	Requirements	30%	3	0.9	Use database-level row locking to safely handle simultaneous requests when the automated first-come-first-served toggle is active	Full-stack developer
R3	Underestimating Development Time Within the Strict Two-Week Schedule	Estimation	50%	2	1.0	Enforce strict MVP scoping—prioritize core Must-have features (Basic Booking and Inventory Management) and treat AI/notifications as secondary	Team Lead
R4	Team Learning Curve Friction with Docker, Django, and PostgreSQL Integration	People	40%	2	0.8	Create a standardized, pre-configured Docker Compose file for local PostgreSQL and environment setups so all members share an identical dev environment	Team Lead
R5	Resistance from Library Staff to Transition from Google Sheets to the New Platform	Organisational	30%	2	0.6	Design the librarian dashboard to mirror familiar table/spreadsheet layouts and provide a concise, step-by-step user guide for staff	Designer

6 Tools & Collaboration Setup

- Course Portal: **Moodle** – announcements and PA submissions.
- Communication Channel: **Discord** is configured as our communication hub.
- Task Management: **GitHub Projects** is set up as our task board.
- Weekly Status Meetings: Held every Tuesday. Weekly status reports are created using the provided template and committed to (/docs/management/weekly) before the weekly deadline.
- Git repository – GitHub: source code and project documentation are hosted on private GitHub repository.
    + Branch Protection: The main branch is protected. Direct pushes are disabled; all changes must be merged via Pull Requests with at least 1 required code review from a peer.
    + Commit Standards: commit messages use the Conventional Commits specification
    + Repository Sanitation: A .gitignore file is configured. Environment variables, API keys, and sensitive credentials are handled in .env files and never committed to the repository.

Repository structure:

Folder	Content
/src	Source code
/docs/management	Project plan; weekly status reports in /docs/management/weekly
/docs/requirements	Vision, SRS, use cases, mockups
/docs/design	Architecture (C4), ADRs, UML models, UI design
/docs/test	Test plan, test cases, test reports
/docs/ai-usage	AI usage logs
/pa/PA1 … /pa/PA5	One sub-folder per PA submission (PDF + source files)

7 AI Usage Log

[Required in every PA (course GenAI policy). List each significant use of AI tools. Write "No AI tools were used" if none. Undisclosed AI use is treated as an academic integrity violation.]

#	Member	Tool	Task / purpose	How the output was verified and changed
1	Nguyễn Võ Nhật Thuận	Gemini	Optimize the WBS structure and allocate tasks in alignment with the schedule for each phase (PA1 – PA5).	Check referenced between the list of tasks and the course rubric requirements; refined the scope of the AI (VLM).
2	Lê Phụng Sơn	Gemini	Construct Preliminary Problem Statement outline from examples provided, then fill the project idea content from PA0 into the outline.	Check the generated text against the original project idea from PA0 to ensure technical accuracy. Remove too academic and filler words, instead use simple words, grammars while maintaining accurate technical terms..
3	Lai Phạm Như Phúc	Gemini	Review, modify, and add missing Work Breakdown Structure (WBS) tasks for sections 1.2 to 1.5 to meet the "Excellent" level criteria in the grading rubric.	Verified against the team's WBS flow. Re-numbered tasks, grouped test execution tasks specifically, and rejected suggestions for component/state diagrams and demo video.
4	Dương Nguyên Nam	Claude / Gemini	Helped understand and draft PA1 sections 2.2 (Product Position Statement), 2.3 (Stakeholders and Users) and 2.4 (Scope and Key Features), contents based on the PA0 Project Idea & Team Contract.	Checked every statement against PA0 (target users, features, similar products, out-of-scope items) and reviewed the draft with the team. Changed feature priorities suggested by AI to match the team's capacity, and edited or removed the features and out-of-scope items suggested by AI that PA0 did not list. Rewrote wording to match the team's terminology.
5	Trần Hoàng Khánh My	Gemini	Draft Preliminary Architecture (C4 Diagram), Tech Stack, and Environment sections for PA1, and translate to English.	Cross-checked the C4 diagram with PA0 scope (confirmed no payment gateway). Kept AI's suggestion for PyTest and GitHub Actions to enhance automated testing.
6	Lai Phạm Như Phúc	Gemini	Review project schedule, Gantt chart, task dependencies, and identify the critical path for Section 4.4 Schedule.	Verified that milestone dates accurately match PA deadlines (Weeks 3, 5, 7, 8, 10, 11). Cross-checked the duration logic to ensure effort hours were realistically spread across calendar days. Added missing effort values

				(task 1.5.1.7) to finalize the critical path.
7	Nguyễn Võ Nhật Thuận	ChatGPT	Used to understand how to use the project rubric to build exit criteria for PA01 Section 4.1.	Compared the suggested exit criteria with the PA01 template and project rubric. Selected criteria to match the required deliverables and submission milestones.
8	Lê Phụng Sơn	Gemini	Help to spot top 5 potential hidden technical and non-technical risks the project might face based on the project idea overview stated in PA0.	Verify the risks against the specific technical constraints of the project. Check that the risks' probability, loss values, and risk exposure calculations are logically consistent, verify the mitigation steps to be practical and actionable.
9	Lai Phạm Như Phúc	Gemini	Calculate human resources cost based on the 56 WBS tasks and draft a concise budget summary (labor, tools, hosting, and contingency).	Verified math totals against WBS hours ($5/hour rate). Decided to remove the labor cost breakdown by project phase.